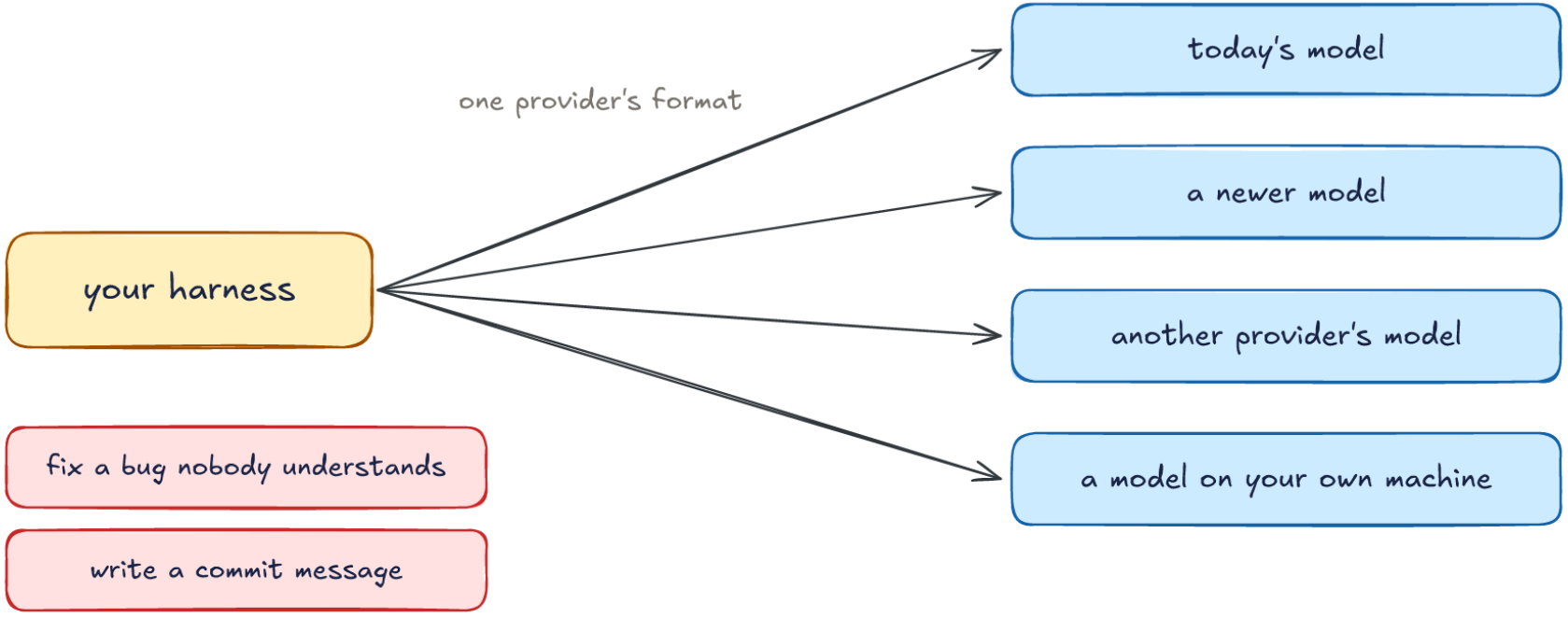
Many Models, One Harness

EECS 498 AASE, Lecture 18

Today's schedule

- [x] Speaking to many models
- [ ] Break
- [x] Choosing a model for each job
- [ ] Questions

A harness built for one model



How does one harness speak to many models, and which does each job?

Part 1

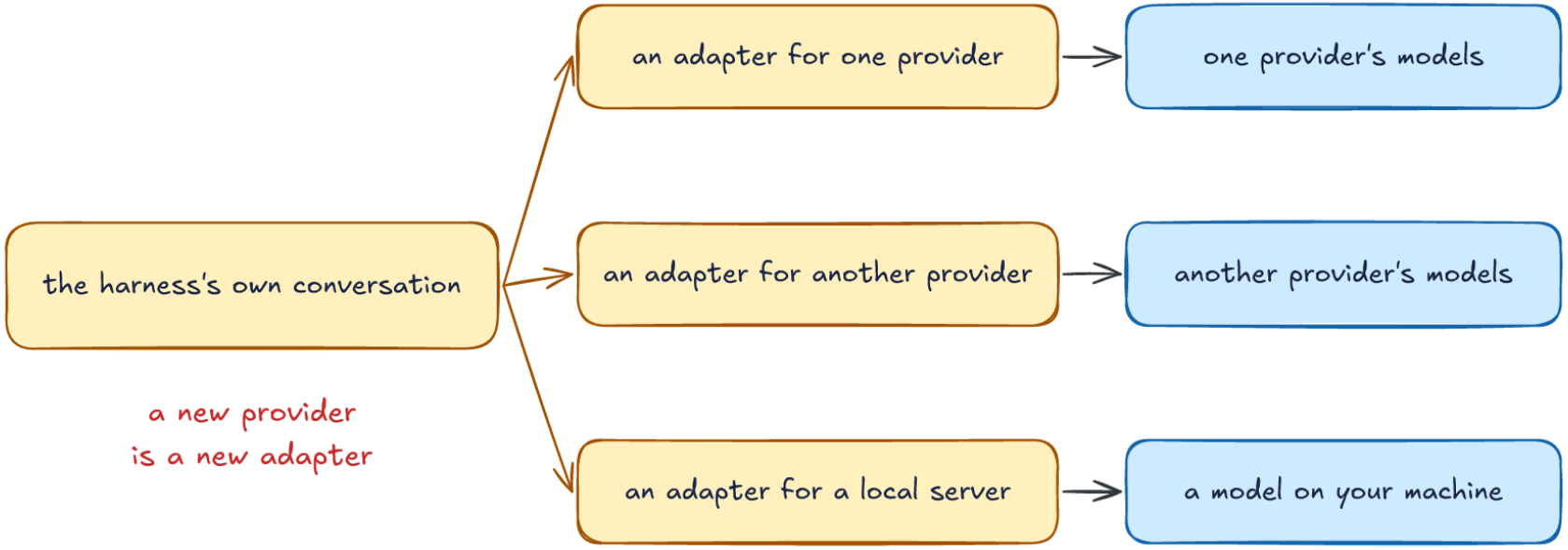
Speaking to many models

One conversation in two shapes

one provider	another provider
the system prompt as the first message	the system prompt in its own field
your message	your message
a tool call with arguments as a string	a tool call block with arguments as an object
the result as a message of its own	the result inside your next message

the harness must build each shape exactly

One format and an adapter per provider



replies and stream events come back in the harness's own shape

Reasoning only one provider can read

a reply



the next request to the same provider



it reads its own reasoning and carries on

the same request to another provider



it cannot read the sealed part, so the adapter leaves it out

the new model reasons again from what it can read

What else differs by provider

	one provider	another provider	the adapter
caching	caches the start by itself	caches only when asked	adds the marks
tool schemas	takes most schemas	rejects some schema parts	rewrites the schema
built-in tools	has a built-in tool	has nothing like it	leaves it out
token counts	includes cached input	lists cached input apart	counts one way

translation carries the conversation but not every feature

A profile for each model



a new model needs a profile, and a new provider also needs an adapter

Chat break

Back in five minutes

Part 2

Choosing a model for each job

One model for every job

the strongest model for every job

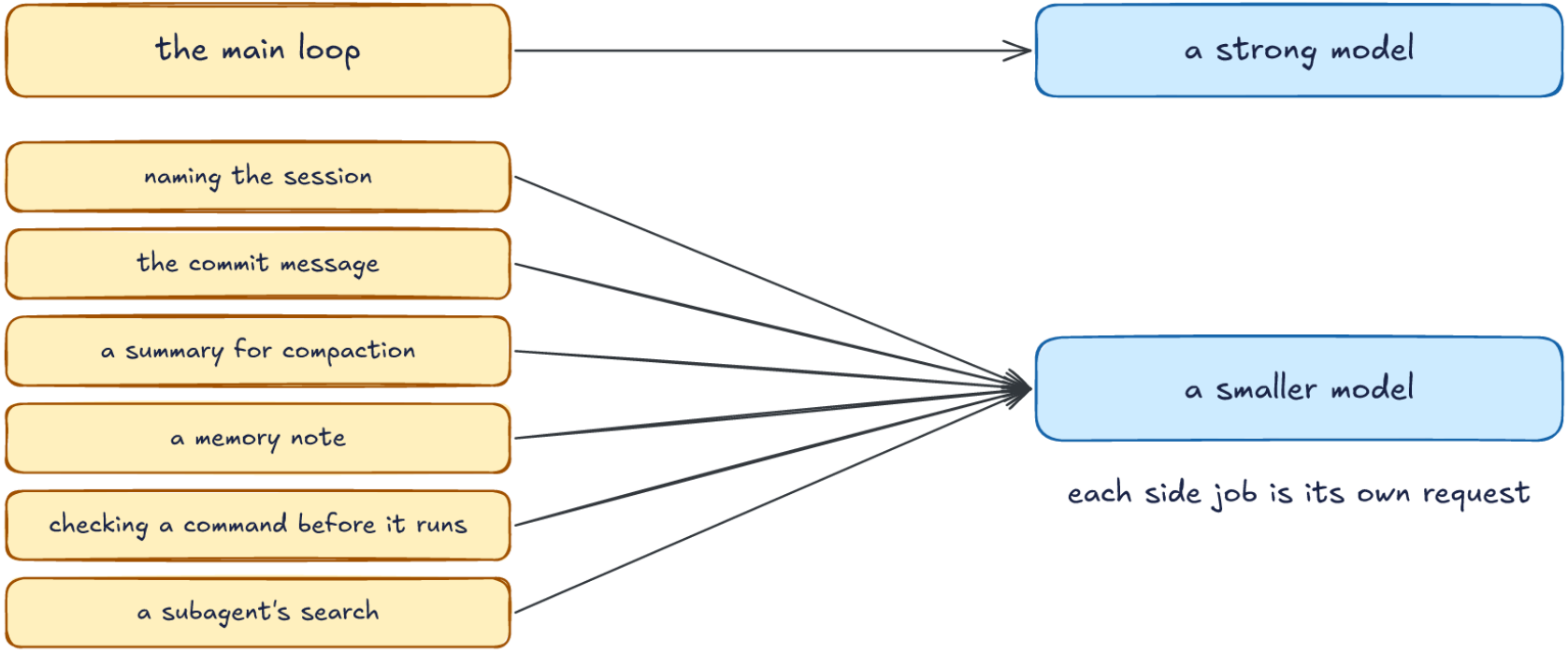
fix a bug nobody understands	name the session	write a commit message
it works	slow and costly	slow and costly

the cheapest model for every job

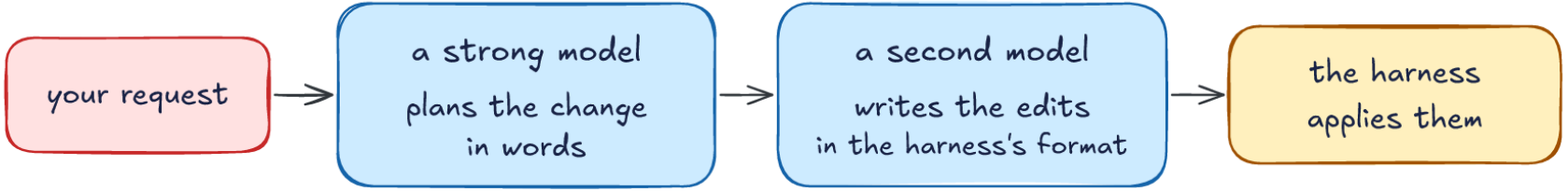
fix a bug nobody understands	name the session	write a commit message
it fails	it works	it works

give each job the cheapest model that does it well

The jobs inside one session



A planner and an editor



each model does the part it is best at

Effort as a dial

low effort
a quick rename



medium effort
an ordinary change



high effort
a bug nobody understands



reasoning is billed like output, so effort costs time and money

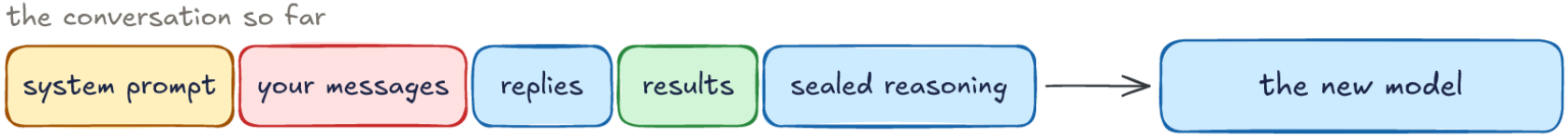
Who picks the model

you	switch models with a command
the harness	a rule sends each job to a model
the main model	it picks a subagent's model
a router	a small model reads the request and picks

Switching models mid-task

you switch models

the harness falls back when one is overloaded



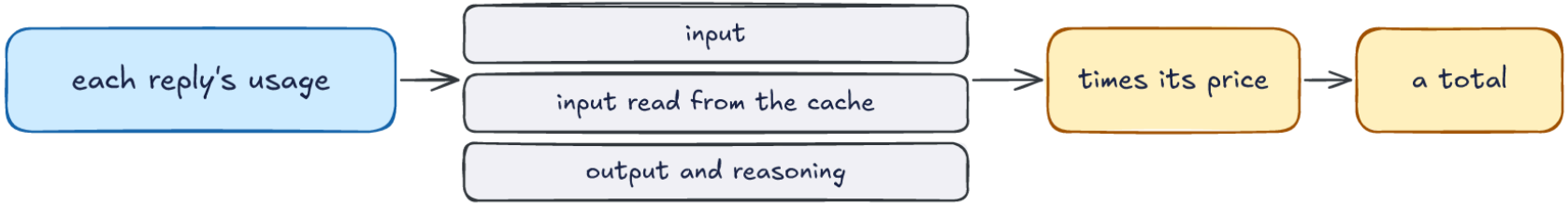
the cache belonged to the old model, so all of it is full price

the sealed reasoning is left out

a smaller window may force compaction first

keep the main model steady and send side jobs to other models

What a session costs



where a long session's money goes



caching makes the repeated input cheaper

a smaller model makes side jobs cheaper

What to take away

- Keep one format and translate at the edge
- Some things do not survive translation
- A profile says how each model works
- Give each job the cheapest model that does it well

This week

- Aider reaches every provider through LiteLLM
- Its model settings are the profile
- Its weak model writes commit messages and summaries

Sources

Anthropic's Messages API
platform.claude.com/docs/en/api/messages

OpenAI on prompt caching
developers.openai.com/api/docs/guides/prompt-caching

RouteLLM on routing between models
arxiv.org/abs/2406.18665

The Aider source
github.com/Aider-AI/aider

Questions

AASE Lecture 18, Many Models, One Harness